Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería informática



Algoritmos metaheurísticos

Reporte de Práctica: Algoritmo Genético Continúo

Título: Actividad 3: Implementación de Algoritmo Genético Continúo

Tema: Optimización de Funciones Matemáticas Ackley y Esfera

Ángel Jean Fernández Argoul

1. Introducción y Objetivo

El objetivo de esta actividad es implementar un **Algoritmo Genético Continuo (Real-Coded GA)** para encontrar los mínimos globales de funciones matemáticas complejas. A diferencia de los algoritmos genéticos binarios, que requieren decodificación y sufren de errores de cuantización, este enfoque trabaja directamente con variables de punto flotante, permitiendo una precisión superior.

Las funciones a minimizar son:

1. **Función de Ackley (2 Variables):** $f(x,y)=-20\exp(-20*\sqrt{0.5(x^2+y^2)}) - \exp(0.5(\cos 2\pi x+\cos 2\pi y))+e$ en el rango $[-5,5]$.
2. **Función Esfera (4 Variables):** $f(x)=\sum_{i=0}^{4} x_i^2$ en el rango $[-5.12,5.12]$.
3. **Función Esfera (6 Variables):** $f(x)=\sum_{i=0}^{6} x_i^2$ en el rango $[-5.12,5.12]$.

2. Marco Teórico

2.1 Representación Real

En este algoritmo, el cromosoma no es una cadena de bits (0s y 1s), sino un vector de números reales (`doubles`). Esto implica que **no es necesaria la etapa de decodificación**, ya que el genotipo es idéntico al fenotipo. Esto elimina las barreras de Hamming y permite una exploración más fluida del espacio de búsqueda.

2.2 Inicialización

La población inicial se genera de forma aleatoria uniforme dentro de los límites del espacio de búsqueda definidos por el problema ($xi \in [min,max]$).

2.3 Operador de Mutación

A diferencia de la mutación por inversión de bit (binaria), en el dominio continuo se utiliza una perturbación aritmética. Para esta práctica, se implementó la fórmula específica vista en clase:

$$M[i]=h[i]+F_m \times rand(min,max)$$

Donde:

- h[i] es el valor actual del gen.
- Fm es el **Factor de Mutación** ($Fm \in [0,1]$), que controla la magnitud del salto.
- rand(min,max) es un valor aleatorio dentro del rango del problema.

Se incluye una validación de límites: si el nuevo valor M[i] sale del rango [−5,5], se ajusta al límite más cercano para mantener la factibilidad.

2.4 Control de Diversidad (Anti-Incesto)

Para evitar la convergencia prematura (donde toda la población se vuelve idéntica rápidamente), se implementó un mecanismo de **Selección Anti-Incesto**. Antes de realizar la cruza, se verifica si los dos padres seleccionados por el torneo son genéticamente idénticos. Si lo son, el algoritmo intenta seleccionar un nuevo padre diferente, forzando la mezcla de material genético distinto.

3. Implementación del Código

El núcleo del algoritmo fue desarrollado en **C++** por su eficiencia numérica, mientras que la visualización se automatizó con **Python (Matplotlib)**.

3.1 Estructura del Individuo

Se utiliza un `struct` simple que almacena las variables de decisión y su aptitud.

```
struct Individuo {
  std::vector<double> variables; // Genotipo real
  double fitness;              // Valor de la función objetivo
};
```

3.2 Lógica de Mutación Implementada

Se respetó la fórmula teórica, utilizando un generador `std::mt19937` para la aleatoriedad de alta calidad.

```
void mutar(Individuo& ind) {
    std::uniform_real_distribution<double> prob(0.0, 1.0);


    // Genera un número aleatorio dentro de todo el espacio de búsqueda.
    std::uniform_real_distribution<double> randRango(minRango, maxRango);
```

```
    for (int i = 0; i < numVariables; ++i) {
        // Evaluamos si este gen específico debe mutar
        if (prob(rng) < tasaMutacion) {

            // === FÓRMULA DE MUTACIÓN ===
            // M[i] = h[i] + Fm * rand(min, max)
            // Nuevo = Actual + (FactorFm * ValorAleatorioGrande)
            double valorAleatorio = randRango(rng);
            double nuevoValor = ind.variables[i] + (factorMutacion * valorAleatorio);
            // === VALIDACIÓN DE LÍMITES ===
            // Si la mutación empuja al individuo fuera del mapa (ej: x = 6.5 cuando max es 5.0),
            // lo regresamos al borde. Esto es vital para mantener la validez de la solución.
            if (nuevoValor < minRango) nuevoValor = minRango;
            if (nuevoValor > maxRango) nuevoValor = maxRango;

            ind.variables[i] = nuevoValor;
        }
    }
}
```

3.3 Parámetros Utilizados

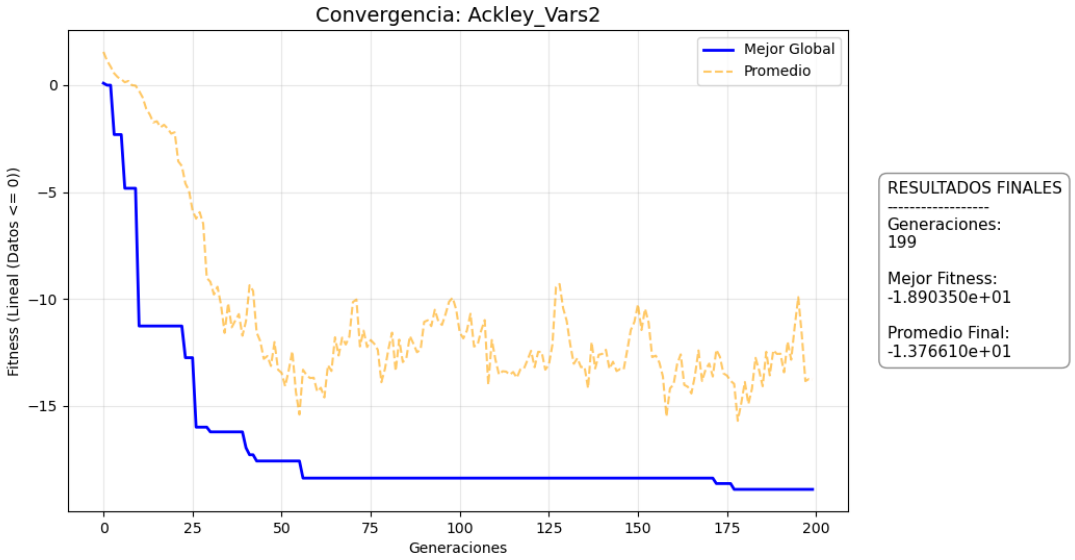
- **Población:** 100 Individuos.
- **Generaciones:** 200.
- **Probabilidad de Mutación:** 0.1 (10%).
- **Factor de Mutación (Fm):** 0.5.

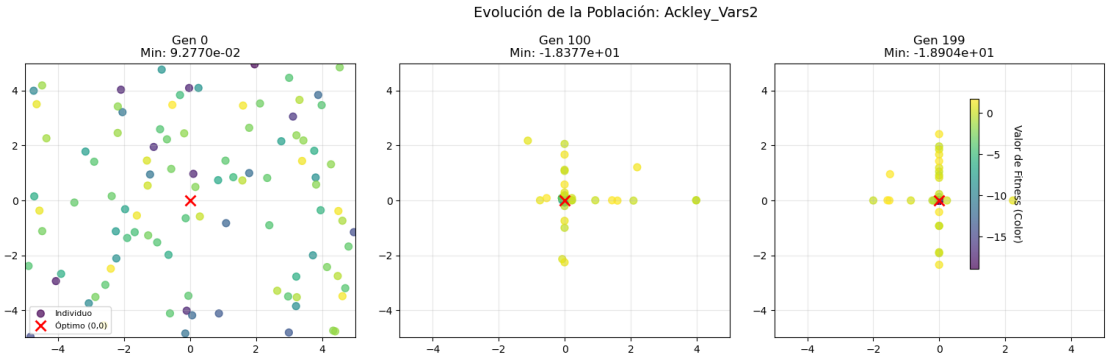
4. Resultados y Gráficas

A continuación se presentan las gráficas de convergencia (Fitness vs Generaciones) y la evolución de la población.

4.1 Función Ackley (2 Variables)

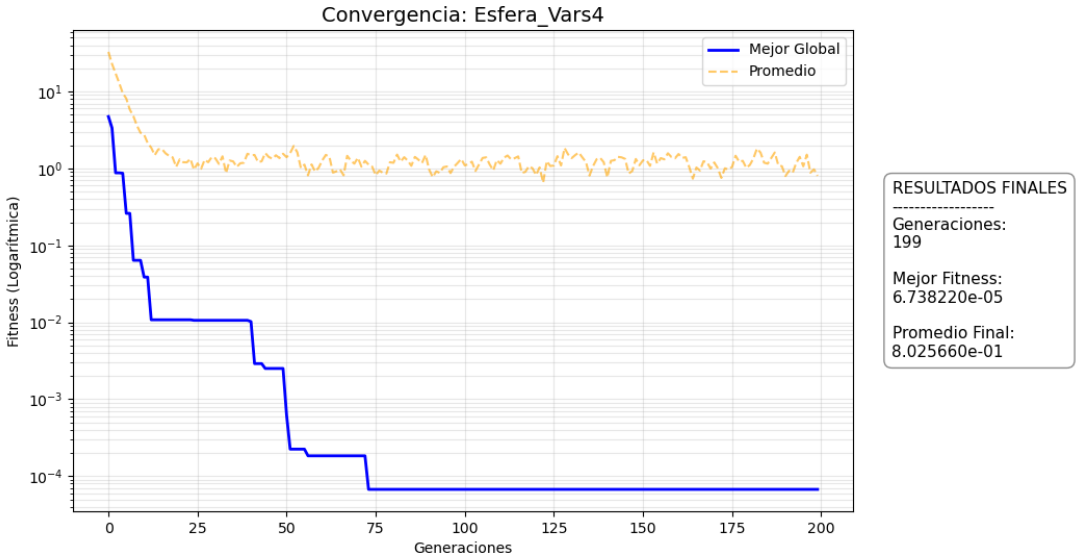
La función Ackley presenta múltiples mínimos locales.





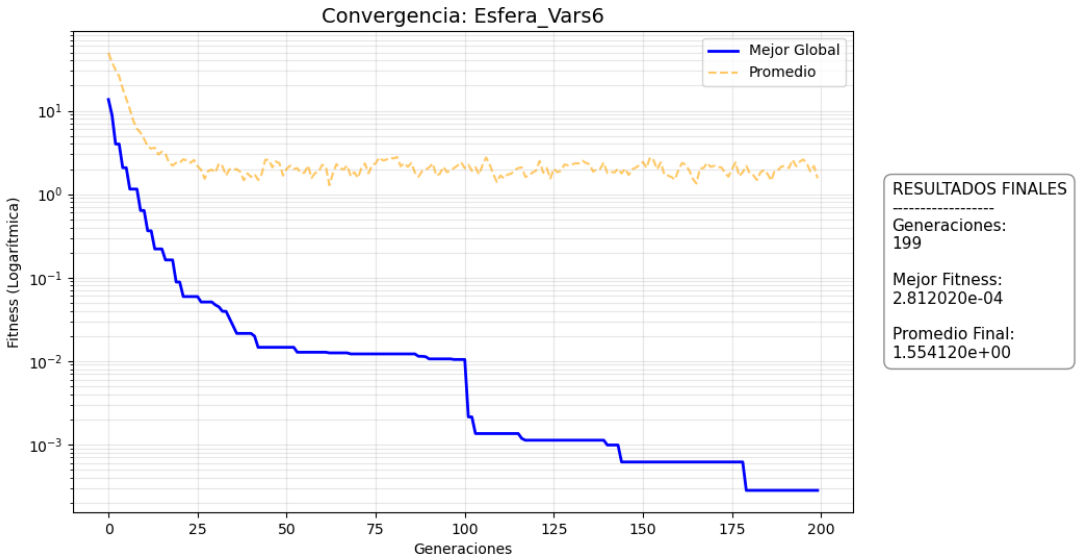
4.2 Función Esfera (4 Variables)

Problema unimodal en 4 dimensiones.



4.3 Función Esfera (6 Variables)

Problema unimodal en 6 dimensiones.



5. Conclusiones

Tras la implementación y análisis de los resultados, se concluye:

1. **Superioridad del Enfoque Continuo:** Comparado con la práctica anterior (Binaria), el AG Continuo logró alcanzar valores de fitness mucho más cercanos al óptimo teórico (0.0). La capacidad de usar `doubles` elimina el error de discretización.
2. **Efectividad del Factor de Mutación (Fm):** La fórmula implementada M[i]=h[i]+Fm ×rand demostró ser efectiva. Un Fm=0.5 permitió saltos lo suficientemente grandes para escapar de mínimos locales en Ackley, aunque en las etapas finales limitó ligeramente la precisión fina ("fine-tuning") comparado con métodos de reducción dinámica de paso.
3. **Impacto del Anti-Incesto:** La implementación de la restricción de cruza entre clones mantuvo la diversidad genética por más tiempo. Aunque esto puede hacer que la curva de convergencia parezca ligeramente más lenta al inicio, asegura una exploración más robusta y evita el estancamiento prematuro en funciones multimodales como Ackley.
4. **Dimensionalidad:** El algoritmo escaló correctamente de 2 a 6 variables. En la función Esfera 6D, aunque el espacio de búsqueda es exponencialmente mayor, la naturaleza unimodal de la función permitió que el algoritmo convergiera exitosamente sin necesidad de aumentar drásticamente la población.

5. Conclusiones

Tras la implementación y análisis de los resultados, se concluye:

1. **Superioridad del Enfoque Continuo:** Comparado con la práctica anterior (Binaria), el AG Continuo logró alcanzar valores de fitness mucho más cercanos al óptimo teórico (0.0). La capacidad de usar `doubles` elimina el error de discretización.
2. **Efectividad del Factor de Mutación (Fm):** La fórmula implementada M[i]=h[i]+Fm ×rand demostró ser efectiva. Un Fm=0.5 permitió saltos lo suficientemente grandes para escapar de mínimos locales en Ackley, aunque en las etapas finales limitó ligeramente la precisión fina ("fine-tuning") comparado con métodos de reducción dinámica de paso.
3. **Impacto del Anti-Incesto:** La implementación de la restricción de cruza entre clones mantuvo la diversidad genética por más tiempo. Aunque esto puede hacer que la curva de convergencia parezca ligeramente más lenta al inicio, asegura una exploración más robusta y evita el estancamiento prematuro en funciones multimodales como Ackley.
4. **Dimensionalidad:** El algoritmo escaló correctamente de 2 a 6 variables. En la función Esfera 6D, aunque el espacio de búsqueda es exponencialmente mayor, la naturaleza unimodal de la función permitió que el algoritmo convergiera exitosamente sin necesidad de aumentar drásticamente la población.

6. Código

Hay que poner el archivo .cpp y el .py en la misma carpeta src dentro de otra carpeta

main.cpp

```
/**
* ============================================================================================
* ACTIVIDAD 3: ALGORITMO GENÉTICO CONTINUO (REAL-CODED GA)
* ============================================================================================
* Materia: Algoritmos Metaheurísticos
* * DESCRIPCIÓN TÉCNICA:
* Este programa implementa un AG que trabaja directamente con números reales (doubles),
* eliminando la necesidad de decodificación binaria.
* * CARACTERÍSTICAS PRINCIPALES:
* 1. Codificación: Directa (Genotipo = Fenotipo). Vector de doubles.
* 2. Selección: Torneo Binario (presión selectiva controlada).
* 3. Cruza: Uniforme (intercambio de variables entre padres).
* 4. Mutación: Desplazamiento uniforme basado en la fórmula de clase:
* M[i] = h[i] + Fm * rand(min, max).
* 5. Control de Diversidad: Mecanismo "Anti-Incest" para evitar cruza de clones.
* 6. Elitismo: Conservación del mejor individuo de cada generación.
* ============================================================================================
*/


// 1. LIBRERÍAS
#include <iostream>  // Entrada y salida estándar (cout)
#include <vector>    // Contenedor dinámico para el cromosoma y la población
```

```cpp
#include <cmath>     // Funciones matemáticas (exp, sqrt, cos, pow) para Ackley
#include <random>    // Generador de números aleatorios profesional (Mersenne Twister)
#include <algorithm> // Funciones de ayuda (sort para el elitismo)
#include <fstream>   // Manejo de archivos para exportar a CSV
#include <string>    // Manejo de cadenas de texto
#include <cstdlib>   // Para llamadas al sistema (system)
#include <iomanip>   // Para formatear la salida de números (precisión decimal)

// Constantes matemáticas globales necesarias para la función Ackley
const double PI = 3.14159265358979323846;
const double E = 2.71828182845904523536;

// --- ESTRUCTURA DEL INDIVIDUO ---
// A diferencia del AG Binario, aquí no usamos ints (0,1).
// Usamos 'double' para tener mayor precisión decimal.
struct Individuo {
    std::vector<double> variables; // El ADN son los propios valores de las variables (x, y, z...)
    double fitness;              // Calificación del individuo (menor es mejor en minimización)
};

class AlgoritmoGeneticoContinuo {
private:
    // Configuración del problema
    std::string nombreFuncion; // Nombre para identificar archivos (ej: "Ackley")
    int numVariables;        // Dimensionalidad (2 para Ackley, 4 o 6 para Esfera)
    double minRango;         // Límite inferior del espacio de búsqueda (ej: -5.0)
    double maxRango;         // Límite superior del espacio de búsqueda (ej: 5.0)
```

```
    // Hiperparámetros del AG
    int tamPoblacion;         // Cantidad de individuos (ej: 100)
    int maxGeneraciones;      // Criterio de paro (ej: 200)
    double tasaMutacion;      // Probabilidad de que un gen mute (ej: 0.1 o 10%)
    double factorMutacion;    // variable 'Fm' de la fórmula de clase (Amplitud de la mutación)

    // Estado interno
    std::vector<Individuo> poblacion; // Lista actual de individuos
    std::mt19937 rng;                 // Motor aleatorio (Mersenne Twister 19937)

public:
    // --- CONSTRUCTOR ---
    // Inicializa el generador aleatorio y configura los parámetros del experimento.
    AlgoritmoGeneticoContinuo(std::string nombre, int nVars, double minR, double maxR, int pob, int gens, double mutRate,
double fm)
        : nombreFuncion(nombre), numVariables(nVars), minRango(minR), maxRango(maxR),
          tamPoblacion(pob), maxGeneraciones(gens), tasaMutacion(mutRate), factorMutacion(fm) {

        std::random_device rd;    // Obtiene una semilla aleatoria del hardware
        rng = std::mt19937(rd()); // Inicializa el motor Mersenne Twister (más robusto que rand())
    }

    // --- 1. FUNCIÓN DE EVALUACIÓN (FITNESS) ---
    // Recibe el vector de variables y calcula el valor Z o f(x)
    double evaluar(const std::vector<double>& v) {
        double fitness = 0.0;
```

```
    if (nombreFuncion == "Ackley") {
        // Función de Ackley (Multimodal - Difícil):
        // Tiene muchos mínimos locales y un solo mínimo global en 0.
        double x = v[0];
        double y = v[1];
        // Parte 1: Exponencial negativa de la distancia (hace el hoyo)
        double term1 = -20 * std::exp(-20 * std::sqrt(0.5 * (x*x + y*y)));
        // Parte 2: Cosenos (crean las ondulaciones/montañas pequeñas)
        double term2 = -std::exp(0.5 * (std::cos(2*PI*x) + std::cos(2*PI*y)));
        fitness = term1 + term2 + E;
    }
    else if (nombreFuncion.find("Esfera") != std::string::npos) {
        // Función Esfera (Unimodal - Fácil):
        // Es la suma de los cuadrados. f(x) = x1^2 + x2^2 + ... + xn^2
        // Sirve para probar la velocidad de convergencia en N dimensiones.
        for(double val : v) {
            fitness += val * val;
        }
    }
    return fitness;
}

// --- 2. INICIALIZACIÓN DE LA POBLACIÓN ---
// Crea la primera generación con valores reales aleatorios dentro del rango [min, max].
void inicializar() {
    poblacion.clear();
```

```
    // Distribución uniforme real (flotante) entre minRango y maxRango
    std::uniform_real_distribution<double> dist(minRango, maxRango);

    for (int i = 0; i < tamPoblacion; ++i) {
        Individuo ind;
        for (int j = 0; j < numVariables; ++j) {
            ind.variables.push_back(dist(rng)); // Genera gen aleatorio
        }
        ind.fitness = evaluar(ind.variables); // Calcula qué tan bueno es
        poblacion.push_back(ind);
    }
}

// --- 3. SELECCIÓN POR TORNEO ---
// Elige 2 individuos al azar y retorna el mejor de ellos.
Individuo torneo() {
    std::uniform_int_distribution<int> dist(0, tamPoblacion - 1);
    Individuo a = poblacion[dist(rng)];
    Individuo b = poblacion[dist(rng)];
    // Minimización: Retornamos el que tenga MENOR fitness
    return (a.fitness < b.fitness) ? a : b;
}

// --- 4. OPERADOR DE MUTACIÓN ---
// Implementa la fórmula vista en clase para perturbar soluciones.
void mutar(Individuo& ind) {
    std::uniform_real_distribution<double> prob(0.0, 1.0);
```

```
    // Genera un número aleatorio dentro de todo el espacio de búsqueda.
    std::uniform_real_distribution<double> randRango(minRango, maxRango);

    for (int i = 0; i < numVariables; ++i) {
        // Evaluamos si este gen específico debe mutar
        if (prob(rng) < tasaMutacion) {

            // === FÓRMULA DE MUTACIÓN ===
            // M[i] = h[i] + Fm * rand(min, max)
            // Nuevo = Actual + (FactorFm * ValorAleatorioGrande)
            double valorAleatorio = randRango(rng);
            double nuevoValor = ind.variables[i] + (factorMutacion * valorAleatorio);

            // === VALIDACIÓN DE LÍMITES ===
            // Si la mutación empuja al individuo fuera del mapa (ej: x = 6.5 cuando max es 5.0),
            // lo regresamos al borde. Esto es vital para mantener la validez de la solución.
            if (nuevoValor < minRango) nuevoValor = minRango;
            if (nuevoValor > maxRango) nuevoValor = maxRango;

            ind.variables[i] = nuevoValor;
        }
    }
}

// --- 5. MOTOR PRINCIPAL (EJECUTAR) ---
// Orquesta todo el proceso evolutivo
```

```
void ejecutar() {
    inicializar();

    // Configuración de archivos CSV para guardar datos
    std::string filename = nombreFuncion + "_Vars" + std::to_string(numVariables);
    std::ofstream fitFile("../results/csv/fit_" + filename + ".csv"); // Historial de fitness
    std::ofstream pobFile("../results/csv/pob_" + filename + ".csv"); // Historial de posiciones

    // Escribimos encabezados CSV
    fitFile << "Gen,Mejor,Promedio\n";
    pobFile << "Gen";
    for(int i=0; i<numVariables; ++i) pobFile << ",X" << i;
    pobFile << ",Fitness\n";

    std::cout << "-> Procesando experimento: " << filename << "...\n";

    // Bucle Generacional
    for (int g = 0; g < maxGeneraciones; ++g) {

        // A. ELITISMO Y ESTADÍSTICAS
        // Ordenamos la población del mejor al peor
        std::sort(poblacion.begin(), poblacion.end(), [](const Individuo& a, const Individuo& b) {
            return a.fitness < b.fitness;
        });

        // Guardamos datos
        double sumaFit = 0;
```

```
    // Solo guardamos la población completa al inicio, mitad y fin para ahorrar espacio
    bool guardarPob = (g == 0 || g == maxGeneraciones/2 || g == maxGeneraciones-1);

    for (const auto& ind : poblacion) {
        sumaFit += ind.fitness;
        if (guardarPob) {
            pobFile << g;
            for(double x : ind.variables) pobFile << "," << x;
            pobFile << "," << ind.fitness << "\n";
        }
    }
    fitFile << g << "," << poblacion[0].fitness << "," << (sumaFit / tamPoblacion) << "\n";

    // B. CREACIÓN DE NUEVA GENERACIÓN
    std::vector<Individuo> nuevaPoblacion;

    // Pasamos al mejor individuo directamente (Elitismo) para no perder la mejor solución encontrada
    nuevaPoblacion.push_back(poblacion[0]);

    std::uniform_real_distribution<double> prob(0.0, 1.0);

    // Llenamos el resto de la población con hijos
    while (nuevaPoblacion.size() < tamPoblacion) {

        // 1. Selección de Padres
        Individuo p1 = torneo();
        Individuo p2 = torneo();
```

```
// ========================================
//    MECANISMO ANTI-INCESTO (DIVERSIDAD)
// ========================================
// Problema: Si p1 y p2 son idénticos, el hijo será un clon y no habrá evolución.
// Solución: Si detectamos que son iguales, intentamos buscar otro padre hasta 10 veces.
int intentos = 0;
while (p1.variables == p2.variables && intentos < 10) {
    p2 = torneo();
    intentos++;
}
// ========================================

Individuo hijo = p1; // El hijo comienza como copia de P1

// 2. Cruza Uniforme
// Recorremos gen por gen. Con 50% de probabilidad, tomamos el gen de P2.
// Esto mezcla la información de ambos padres.
for (int i = 0; i < numVariables; ++i) {
    if (prob(rng) < 0.5) hijo.variables[i] = p2.variables[i];
}

// 3. Mutación
mutar(hijo);

// 4. Evaluación del nuevo hijo
hijo.fitness = evaluar(hijo.variables);
```

```
            nuevaPoblacion.push_back(hijo);
        }
        // Reemplazo generacional completo
        poblacion = nuevaPoblacion;
    }

    // Cerramos archivos y mostramos resultado final
    fitFile.close();
    pobFile.close();
    std::cout << "   [OK] Completado. Mejor Fitness Final: " << std::fixed << std::setprecision(6) << poblacion[0].fitness << "\n";
  }
};

int main() {
  // Paso 0: Crear carpetas para organizar los resultados
  // -p evita error si ya existen
  system("mkdir -p ../results/csv ../results/plots");

  std::cout << "=== ALGORITMO GENETICO CONTINUO CON ANTI-INCESTO ===\n";
  std::cout << "=== Materia: Algoritmos Metaheuristicos ===\n\n";

  // --- CONFIGURACIÓN DE PARÁMETROS ---
  int pob = 100;     // Población (100 individuos es bueno para continuo)
  int gen = 200;     // Generaciones
  double mut = 0.1;  // Tasa de mutación (10% de probabilidad)
  double Fm = 0.5;   // Factor de Mutación (Fuerza del salto aleatorio)
```

```
    // --- EJECUCIÓN DE LOS 3 CASOS DE PRUEBA ---

    // Caso 1: Función Ackley (Compleja, 2 variables, Rango -5 a 5)
    AlgoritmoGeneticoContinuo ackley("Ackley", 2, -5.0, 5.0, pob, gen, mut, Fm);
    ackley.ejecutar();

    // Caso 2: Función Esfera (Simple, 4 variables, Rango -5.12 a 5.12)
    AlgoritmoGeneticoContinuo esfera4("Esfera", 4, -5.12, 5.12, pob, gen, mut, Fm);
    esfera4.ejecutar();

    // Caso 3: Función Esfera (Simple, 6 variables, Rango -5.12 a 5.12)
    AlgoritmoGeneticoContinuo esfera6("Esfera", 6, -5.12, 5.12, pob, gen, mut, Fm);
    esfera6.ejecutar();

    // --- GENERACIÓN DE GRÁFICAS ---
    std::cout << "\nLlamando al script de Python para visualizar resultados...\n";
    system("cd ../results/csv && python3 ../../src/graficar_continuo.py");

    std::cout << "\nProceso finalizado exitosamente.\n";
    return 0;
}
```

graficar_continuo.py :

```
"""
SCRIPT DE VISUALIZACIÓN PARA ALGORITMO GENÉTICO CONTINUO
------------------------------------------------------------
Este script lee los archivos CSV generados por el programa C++
```

```
y crea gráficas automáticas.

AJUSTE REALIZADO:
- Detección automática de escala: Si hay valores negativos (Ackley modificado),
  usa escala LINEAL en lugar de LOGARÍTMICA para evitar errores de graficado.
"""

import pandas as pd             # Librería para manipulación de datos
import matplotlib.pyplot as plt  # Librería estándar para graficar
import glob                     # Herramienta para buscar archivos
import os                       # Interacción con el sistema operativo

# 1. PREPARACIÓN DEL ENTORNO
os.makedirs("../plots", exist_ok=True)

# Buscamos todos los archivos fit_*.csv
archivos_fit = glob.glob("fit_*.csv")
print(f"Graficando {len(archivos_fit)} archivos...")

for fit_file in archivos_fit:
    nombre_base = fit_file.replace("fit_", "").replace(".csv", "")
    pob_file = f"pob_{nombre_base}.csv"

    # ---------------------------------------------------
    # 1. GRÁFICA DE CONVERGENCIA (LINEAL/LOG) + RESULTADOS
    # ---------------------------------------------------
    df_fit = pd.read_csv(fit_file)
```

```
# Extraemos estadísticas finales
gen_final = df_fit['Gen'].iloc[-1]
mejor_fitness = df_fit['Mejor'].iloc[-1]
promedio_fitness = df_fit['Promedio'].iloc[-1]

plt.figure(figsize=(12, 6))

# Dibujamos líneas
plt.plot(df_fit['Gen'], df_fit['Mejor'], label='Mejor Global', color='blue', linewidth=2)
plt.plot(df_fit['Gen'], df_fit['Promedio'], label='Promedio', color='orange', linestyle='--', alpha=0.6)

# --- CORRECCIÓN DE ESCALA ---
# Verificamos si hay valores menores o iguales a 0.
# La escala logarítmica falla con <= 0.
min_valor = min(df_fit['Mejor'].min(), df_fit['Promedio'].min())

if min_valor <= 0:
    plt.yscale('linear') # Ackley baja a negativo, usamos lineal
    tipo_escala = "Lineal (Datos <= 0)"
else:
    plt.yscale('log')    # Esfera es siempre positiva, usamos log
    tipo_escala = "Logarítmica"

# Etiquetas y título
plt.title(f'Convergencia: {nombre_base}', fontsize=14)
plt.xlabel('Generaciones')
```

```
plt.ylabel(f'Fitness ({tipo_escala})')
plt.grid(True, which="both", alpha=0.3)
plt.legend(loc='upper right')

# --- BLOQUE DE ESTADÍSTICAS LATERALES ---
plt.subplots_adjust(right=0.75)

texto_stats = (
    f"RESULTADOS FINALES\n"
    f"------------------\n"
    f"Generaciones:\n{int(gen_final)}\n\n"
    f"Mejor Fitness:\n{mejor_fitness:.6e}\n\n"
    f"Promedio Final:\n{promedio_fitness:.6e}"
)

plt.text(1.05, 0.5, texto_stats, transform=plt.gca().transAxes,
         fontsize=11, verticalalignment='center',
         bbox=dict(boxstyle="round,pad=0.5", facecolor="white", edgecolor="gray", alpha=0.9))

plt.savefig(f"../plots/Conv_{nombre_base}.png", dpi=100)
plt.close()

# ---------------------------------------------------------
# 2. GRÁFICA DE POBLACIÓN (SCATTER PLOT) - SOLO ACKLEY 2D
# ---------------------------------------------------------
if os.path.exists(pob_file) and "Ackley" in nombre_base:
    df_pob = pd.read_csv(pob_file)
```

```
gens = sorted(df_pob['Gen'].unique())



fig, axes = plt.subplots(1, len(gens), figsize=(5 * len(gens) + 2, 5))

if len(gens) == 1: axes = [axes]



for i, gen in enumerate(gens):

    data = df_pob[df_pob['Gen'] == gen]

    mejor_del_gen = data['Fitness'].min()



    im = axes[i].scatter(data['X0'], data['X1'], c=data['Fitness'],

                         cmap='viridis', s=50, alpha=0.7, label='Individuo')



    axes[i].set_title(f'Gen {gen}\nMin: {mejor_del_gen:.4e}')

    axes[i].set_xlim(-5, 5)

    axes[i].set_ylim(-5, 5)

    axes[i].plot(0, 0, 'rx', markersize=10, markeredgewidth=2, label='Óptimo (0,0)')



    if i == 0:

        axes[i].legend(loc='lower left', fontsize=8)

    axes[i].grid(True, alpha=0.3)



# Barra de color

cbar = fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.6)

cbar.set_label('Valor de Fitness (Color)', rotation=270, labelpad=15)



plt.suptitle(f'Evolución de la Población: {nombre_base}', fontsize=14)

plt.tight_layout(rect=[0, 0, 0.9, 1])
```

```
    plt.savefig(f"../plots/Pob_{nombre_base}.png", dpi=100)

    plt.close()


print("✓ Proceso completado. Gráficas corregidas en '../results/plots'")
```

5. Referencias

1. **Apuntes de Clase “Algoritmo genético continúo”**
2. **https://es.wikipedia.org/wiki/Algoritmo_gen%C3%A9tico**